

阅前须知

声明

北京合众恒跃科技有限公司保留随时对其产品进行修正、改进和完善的权利，客户在下单前应获取相关信息的最新版本，并验证这些信息是正确的。本文档一切解释权归北京合众恒跃科技有限公司所有。

简介

北京合众恒跃科技有限公司位于中关村科技园区，公司主要从事嵌入式产品的设计、研发、生产、销售等业务；公司的核心竞争力是对嵌入式产品精准的设计及实现能力，成本控制能力和产品品质保障能力；公司主干研发人员均有多年嵌入式产品开发经验和软硬件技术积累，服务于音视频、图像识别、电力等多个应用领域，公司研发设计的视频转码卡、图像识别卡、电力保护装置等产品在业内享有一定的知名度。

联系方式

北京合众恒跃科技有限公司

电话：010-62129511

邮编：102208

技术支持邮箱：support@hzhytech.com

地址：北京市海淀区安宁庄后街南 1 号 A 区一层 1020 号



官方微信公众号



官方企业微信



合众嵌入式官方店铺



合众 AI 官方店铺

修改记录

版本号	日期	修改人	备注
1.0	2024-04-17	李园园	初始版本

目录

阅前须知	2
声明	2
简介	2
联系方式	2
修改记录	3
目录	4
第 1 章 概述	6
1.1 目标	6
1.2 平台差异	6
第 2 章 环境搭建	7
2.1 环境准备	7
2.2 工具链检查	8
2.3 依赖库安装	8
2.4 Qt Creator 安装	9
第 3 章 tslib 编译与部署	15
3.1 tslib 概述	15
3.2 tslib 编译	15
3.3 tslib 部署与测试	17
第 4 章 Qt 编译与部署	21
4.1 Qt 概述	21
4.2 Qt 交叉编译	21
4.3 Qt 部署	24
第 5 章 Qt 应用开发	26

5.1 概述	26
5.2 Qt Creator 使用	26
5.3 Qt 示例演示	37

第 1 章 概述

1.1 目标

本手册面向使用我司各款 ARM 架构开发板的用户，提供从环境搭建到 Qt 应用开发的完整指导。阅读本手册后，您将能够：

- 逐步完成交叉编译工具链和依赖库的安装与配置；
- 成功编译并部署 tslib 和 Qt 库；
- 在 Qt Creator 中创建并编译嵌入式应用；
- 结合示例演示，快速掌握应用开发流程。

本手册内容以通用的 ARM 架构嵌入式 Linux 环境为基础，示例和步骤适用于多种开发板，方便您根据实际硬件环境灵活应用。

1.2 平台差异

在将本手册应用于不同 ARM 架构开发板时，移植 Qt 和 tslib 过程中主要存在以下差异：

- 交叉编译工具链

交叉编译工具链需根据具体目标设备的硬件架构和系统环境选择和配置，确保兼容性和正确性。

- tslib 事件设备节点

tslib 默认读取的触摸事件设备节点因设备而异，通常位于 `/dev/input/eventX` 路径下。使用时请根据实际设备的触摸输入节点进行确认和配置。

- Qt 帧缓冲设备节点

Qt 渲染所使用的帧缓冲设备路径因设备不同而有所区别，通常为 `/dev/fbX`。请根据目标设备的显示硬件环境确认正确的帧缓冲设备路径。

以上差异需结合具体硬件平台实际情况进行调整，以确保 Qt 和 tslib 的正常运行。

第 2 章 环境搭建

2.1 环境准备

在开始编译之前，需要在开发主机（通常为 x86_64 架构的 Ubuntu 系统）上配置基本的软件依赖和相关工具。推荐使用 Ubuntu 18.04 / 20.04 / 22.04 LTS 版本作为开发主机操作系统。

推荐主机环境：

表 2.2-1 主机环境

项目	说明
主机系统	Ubuntu 20.04 LTS（建议使用 64 位版本）
内存	$\geq 2\text{GB}$
硬盘空间	$\geq 100\text{GB}$
网络	可访问外部网络（获取依赖包）
用户权限	拥有 sudo 权限
交叉编译工具链	构建适用于 ARM 架构的应用程序

有关主机开发环境的搭建，请参照随板卡提供的资料包内《3.文档教程/开发板使用手册/Linux 开发环境搭建》手册进行操作。

源代码的获取：当前采用的是 tslib1.16 版本和 qt5.14.2 版本。若需其他版本，请自行进行获取。

表 2.2-2 资料获取

资料	下载链接	备注
Tslib1.16	百度网盘下载链接	百度网盘提取码: hzhy
	官网下载链接	官网
Qt5.14.2	百度网盘下载链接	百度网盘提取码: hzhy
	官网下载链接	官网

2.2 工具链检查

交叉编译工具链的安装与配置在《linux 开发环境搭建手册》中有详细说明，用户可参考该手册完成工具链的配置和检查。

2.3 依赖库安装

执行以下命令以安装编译所需的依赖库。

```
hzhy@ubuntu:$ sudo apt update
hzhy@ubuntu:$ sudo apt install -y \build-essential \flex \bison \gperf \cmake \ninja-build \libssl-dev \autoconf \libtool \libtool-bin
```


2.4 Qt Creator 安装

1) 在 Qt 官网或者国内的 Qt 镜像站下载 Qt Creator 安装包（建议使用较新的版本，界面友好）。

官网下载地址：[下载链接](#)，根据需要选择合适的版本（官网下载缓慢，建议使用国内镜像源下载）。

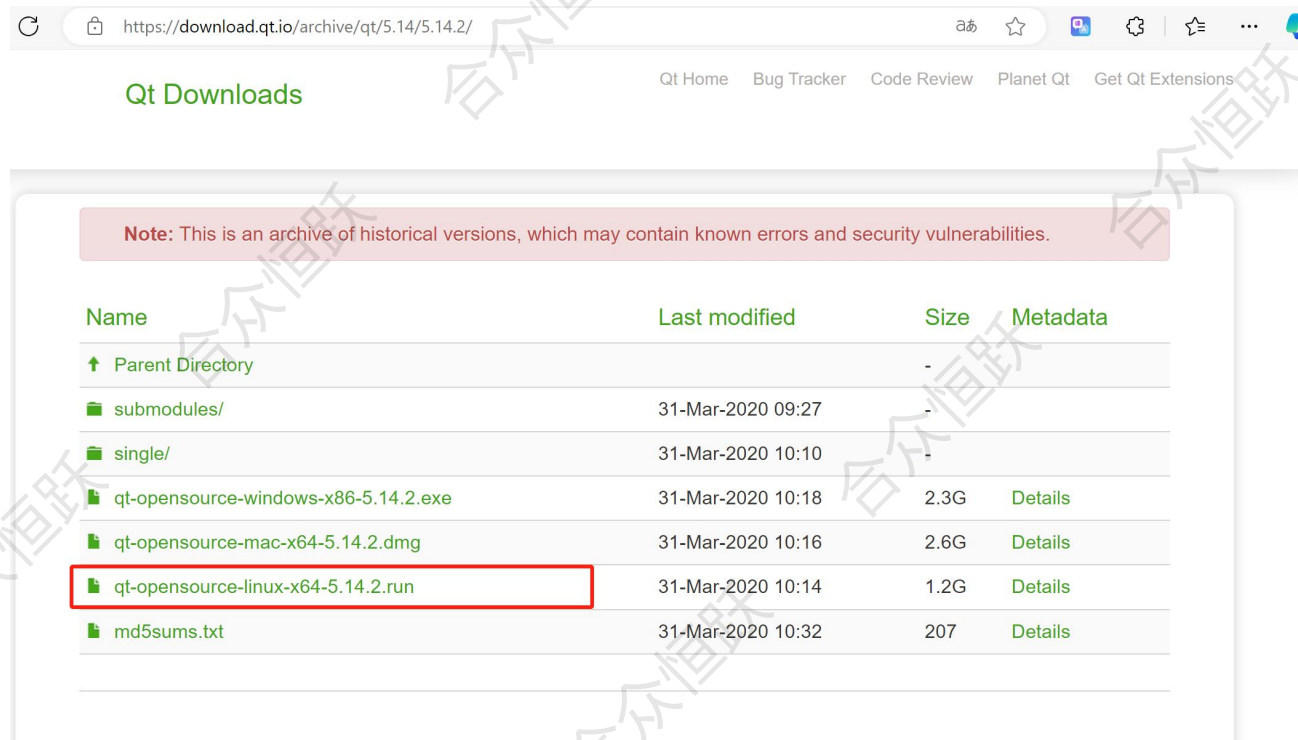


图 2.4-1 安装包下载

镜像站下载地址：[下载链接](#)，根据需要选择合适版本，建议使用 6.0 以上或更高版本，如果无法下载请尝试其它镜像源下载。



图 2.4-2 安装包下载

- 2) 将下载的安装包放置到 ubuntu 系统的/home/Qt_App 目录。

```
hzy@ubuntu:~/Qt_App$ ls
qt-opensource-linux-x64-5.14.2.run
```

图 2.4-3 文件传输

- 3) 给安装包增加可执行权限。

```
hzy@ubuntu:~/Qt_App$ sudo chmod a+x qt-opensource-linux-x64-5.14.2.run
[sudo] hzy 的密码:
hzy@ubuntu:~/Qt_App$ ls -l qt-opensource-linux-x64-5.14.2.run
-rwxr-xr-x 1 root root 1335706944 4月 28 14:11 qt-opensource-linux-x64-5.14.2.run
```

图 2.4-4 增加权限

- 4) 在 ubuntu 中找到该文件，然后双击进行安装（安装之前先断开网络连接，后面需要验证账号）。

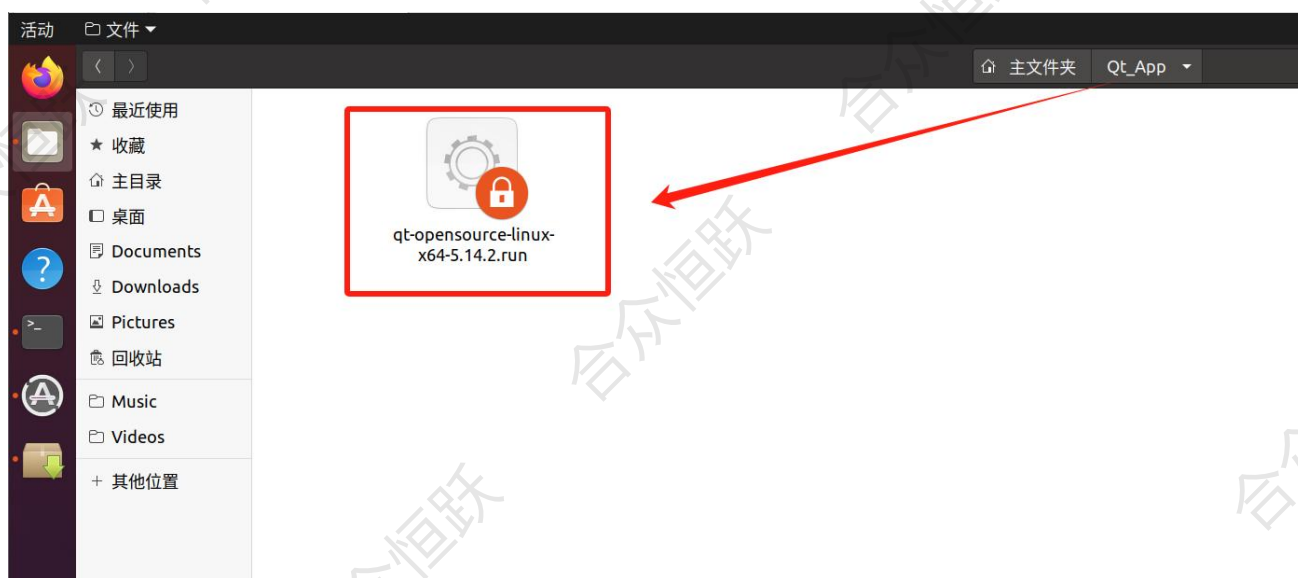


图 2.4-5 开始安装

- 5) 点击 next。

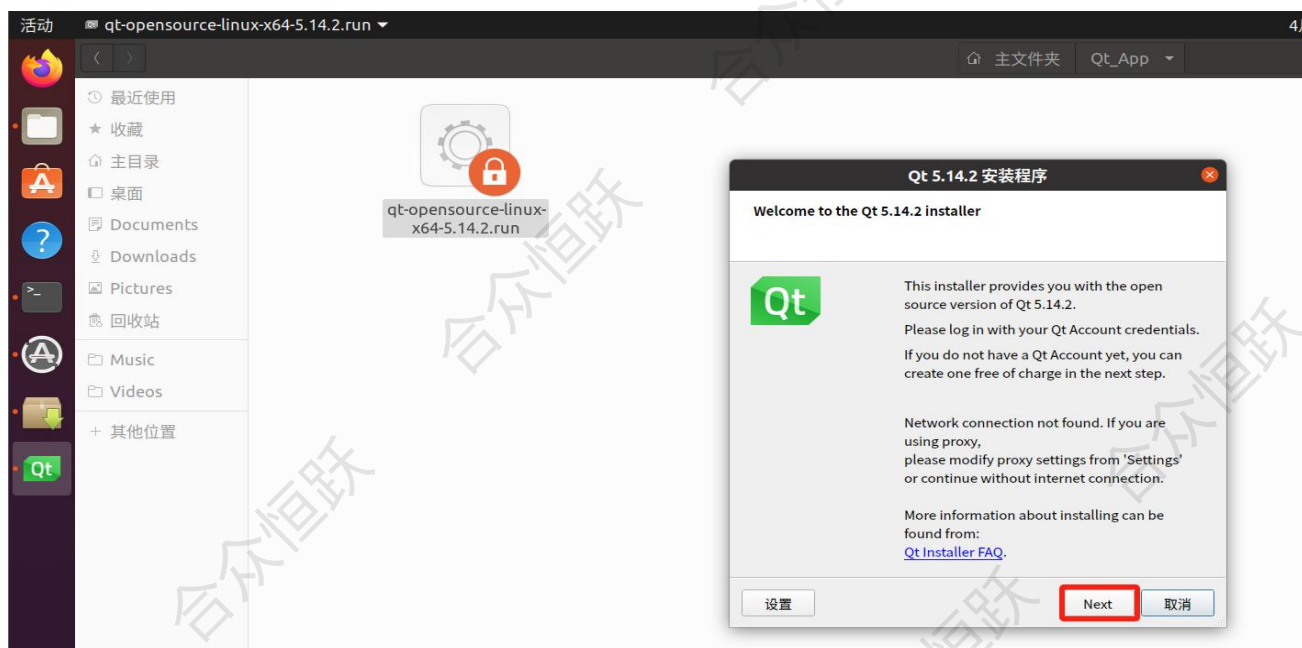


图 2.4-6 点击下一步

6) 指定安装目录，用户可根据自己情况选择合适的安装目录。



图 2.4-7 安装目录

7) 选择需要安装的组件，这里只选择如下一个选项，只安装开发工具，然后点击下一步（如果安装了其它版本也只勾选这个选项，目前我们只安装开发工具，不需要其它多余的组件）。



图 2.4-8 组件选择

8) 接受许可协议，然后继续下一步。

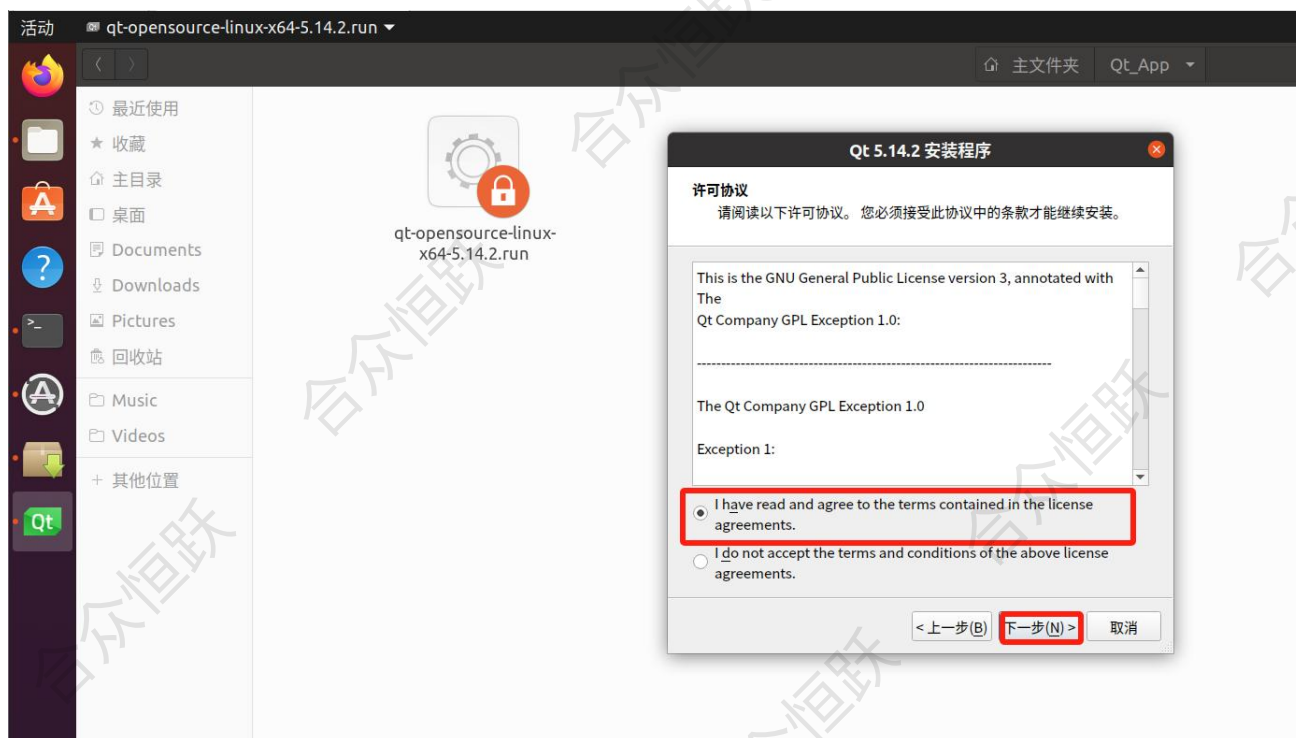


图 2.4-9 接受许可

9) 点击安装。



图 2.4-10 点击安装

10) 安装完成,勾选 Launch Qt Creator 打开 Qt Creator(如果勾选了,安装完成会自动打开 Qt Creator)。



图 2.4-11 安装完成

11) 也可以在应用程序菜单中双击打开,如下所示。



图 2.4-12 打开 Qt Creator

12) 打开之后的主界面如下所示（不同版本界面布局可能有一定差异，使用方法一致）。



图 2.4-13 主界面

第 3 章 tslib 编译与部署

3.1 tslib 概述

在使用触摸屏的嵌入式系统中，**tslib** 是一个常见的轻量级触摸屏抽象层库，提供触摸校准、事件过滤与设备统一接口功能。若目标平台的驱动不支持标准输入事件格式（如 `/dev/input/eventX`），或需要实现更精确的触摸控制，则推荐编译并使用 **tslib**。

编译目的与适用场景

使用 **tslib** 的主要目的：

- 支持触摸屏校准（如 `ts_calibrate` 工具）
- 提供统一的输入事件接口，简化 Qt 等上层应用的适配
- 过滤抖动、缩放等低层触摸异常数据

适用场景：

- 使用电阻触摸屏或原始输入设备
- Qt 使用 linux input 驱动或 **tslib** 插件进行触摸支持
- 无法直接使用 `/dev/input/eventX` 的多点触摸设备

3.2 tslib 编译

1) 请参照第 2.1 节内容下载 **tslib1.16** 的源码包，并将其放置于 Ubuntu 系统中的 `/home/Qt_App/` 目录下。随后，执行以下命令对其进行解压。

```
hzhy@ubuntu:~/Qt_App$ pwd
/home/hzhy/Qt_App
hzhy@ubuntu:~/Qt_App$ ls
tslib-1.16.tar.gz
hzhy@ubuntu:~/Qt_App$ tar -xvf tslib-1.16.tar.gz
```

图 3.2-1 解压源码包

- 2) 进入解压后的 tslib-1.16 源码目录，执行如下命令配置 tslib 并生成 Makefile 文件。

```
hzhy@ubuntu:~/Qt_App$ cd tslib-1.16/
hzhy@ubuntu:~/Qt_App/tslib-1.16$ ./autogen.sh
libtoolize: putting auxiliary files in '.'.
libtoolize: copying file './ltmain.sh'
libtoolize: putting macros in AC_CONFIG_MACRO_DIRS, 'm4/internal'.
libtoolize: copying file 'm4/internal/libtool.m4'
libtoolize: copying file 'm4/internal/ltoptions.m4'
libtoolize: copying file 'm4/internal/ltugar.m4'
libtoolize: copying file 'm4/internal/ltversion.m4'
libtoolize: copying file 'm4/internal/ltobsolete.m4'
configure.ac:58: installing './compile'
configure.ac:7: installing './missing'
plugins/Makefile.am: installing './depcomp'
```

图 3.2-2 生成 Makefile

- 3) 创建安装路径，执行如下命令对 tslib 进行配置。

```
./configure --host=aarch64-none-linux-gnu --prefix=/home/hzhy/Qt_App/tslib-1.16/_install/
CC=aarch64-none-linux-gnu-gcc CXX=aarch64-none-linux-gnu-g++ --cache-file=aarch64.cache
```

```
hzhy@ubuntu:~/Qt_App/tslib-1.16$ mkdir _install
hzhy@ubuntu:~/Qt_App/tslib-1.16$ ./configure --host=aarch64-none-linux-gnu --pre
fix=/home/hzhy/Qt_App/tslib-1.16/_install/ CC=aarch64-none-linux-gnu-gcc CXX=aar
ch64-none-linux-gnu-g++ --cache-file=aarch64.cache
```

图 3.2-3 配置 tslib

- 4) 执行如下命令，编译 tslib 源码（j4：四线程编译，根据实际电脑配置选择）。

```
hzhy@ubuntu:~/Qt_App/tslib-1.16$ make -j4
make all-recursive
make[1]: 进入目录"/home/hzhy/Qt_App/tslib-1.16"
Making all in etc
make[2]: 进入目录"/home/hzhy/Qt_App/tslib-1.16/etc"
make[2]: 对"all"无需做任何事。
make[2]: 离开目录"/home/hzhy/Qt_App/tslib-1.16/etc"
Making all in src
make[2]: 进入目录"/home/hzhy/Qt_App/tslib-1.16/src"
CC      ts_attach.lo
CC      ts_close.lo
```

图 3.2-4 源码编译

- 5) 打包安装，将生成的相关文件安装到 _install 目录。


```

hzhy@ubuntu:~/Qt_App/tslib-1.16$ make install
Making install in etc
make[1]: 进入目录"/home/hzhy/Qt_App/tslib-1.16/etc"
make[2]: 进入目录"/home/hzhy/Qt_App/tslib-1.16/etc"
/usr/bin/mkdir -p '/home/hzhy/Qt_App/tslib-1.16/_install/etc'
/usr/bin/install -c -m 644 ts.conf '/home/hzhy/Qt_App/tslib-1.16/_install/etc'
make[2]: 对"install-data-am"无需做任何事。
make[2]: 离开目录"/home/hzhy/Qt_App/tslib-1.16/etc"
make[1]: 离开目录"/home/hzhy/Qt_App/tslib-1.16/etc"
Making install in src

```

图 3.2-5 打包安装

6) 查看生成的文件

```

hzhy@ubuntu:~/Qt_App/tslib-1.16$ tree -d _install/
_install/
├── bin
├── etc
├── include
├── lib
│   ├── pkgconfig
│   └── ts
├── share
│   └── man
│       ├── man1
│       ├── man3
│       └── man5

```

图 3.2-6 查看生成文件

执行到这一步，tslib 就交叉编译完成。

3.3 tslib 部署与测试

1) 将上述编译生成的文件放置到开发板/opt 目录并重命名为 tslib。

```

root@HZHY:~# cp _install/ /opt/tslib -r
root@HZHY:~# cd /opt/
root@HZHY:/opt# ls tslib/
bin  etc  include  lib  share

```

图 3.3-1 文件传输

2) 通过 vi 命令打开/etc/profile，添加如下内容配置 tslib 环境变量。

```

export TSLIB_ROOT=/opt/tslib
export TSLIB_TSDEVICE=/dev/input/event1
export TSLIB_CONFFILE=$TSLIB_ROOT/etc/ts.conf

```

```
export TSLIB_CALIBFILE=$TSLIB_ROOT/etc/ts.calib
export TSLIB_PLUGINDIR=$TSLIB_ROOT/lib/ts
export TSLIB_FBDEVICE=/dev/fb0
export LD_LIBRARY_PATH=$TSLIB_ROOT/lib:$LD_LIBRARY_PATH
export PATH=$TSLIB_ROOT/bin:$PATH
```

```
export TSLIB_ROOT=/opt/tslib
export TSLIB_TSDEVICE=/dev/input/event1
export TSLIB_CONFFILE=$TSLIB_ROOT/etc/ts.conf
export TSLIB_CALIBFILE=$TSLIB_ROOT/etc/ts.calib
export TSLIB_PLUGINDIR=$TSLIB_ROOT/lib/ts
export TSLIB_FBDEVICE=/dev/fb0
export LD_LIBRARY_PATH=$TSLIB_ROOT/lib:$LD_LIBRARY_PATH
export PATH=$TSLIB_ROOT/bin:$PATH
```

图 3.3-2 环境配置

3) 执行如下命令使配置生效。

```
root@HZHY:/opt# source /etc/profile
```

图 3.3-3 环境配置

4) 执行如下命令进行触摸屏校准，它会显示一个五点校准界面，让你用手依次点一下屏幕上的十字。

```
root@HZHY:/opt/tslib/bin# ./ts_calibrate
xres = 1024, yres = 600
Took 1 samples ...
Top left : X = 73 Y = 73
Took 1 samples ...
Top right : X = 961 Y = 45
Took 1 samples ...
Bot right : X = 955 Y = 550
Took 1 samples ...
Bot left : X = 58 Y = 553
Took 1 samples ...
Center : X = 505 Y = 293
-23.082130 1.035587 0.021527
-16.076267 0.017453 1.014427
Calibration constants: -1512710 67868 1410 -1053574 1143 66481 65536
```

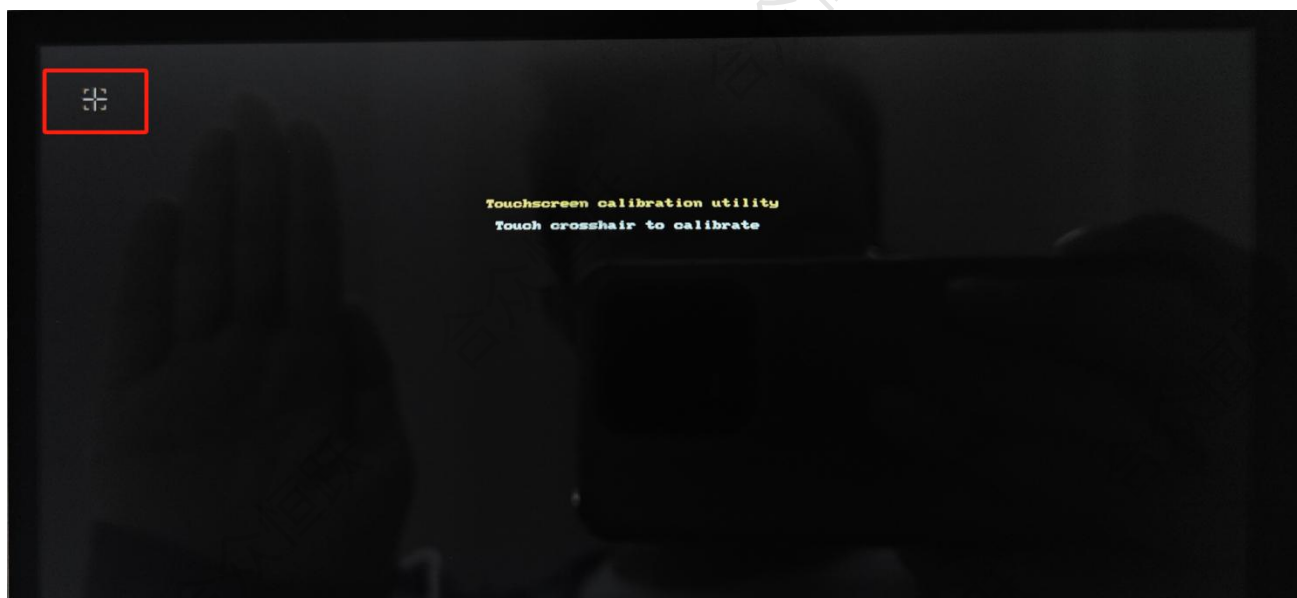


图 3.3-4 触摸屏校准

5) 测试 tslib 能否正常读取触摸事件，执行程序可以在屏幕上画线。

```
root@HZHY:/opt/tslib/bin# ./ts_test
1745807252.946170:    514        5    255
1745807253.158186:    514        7    255
1745807253.169914:    514        9     0
1745807253.843859:    538       28    255
1745807253.973310:    538       28     0
1745807255.203378:    394      358    255
1745807255.250024:    396      358    255
1745807255.261973:    399      358    255
```

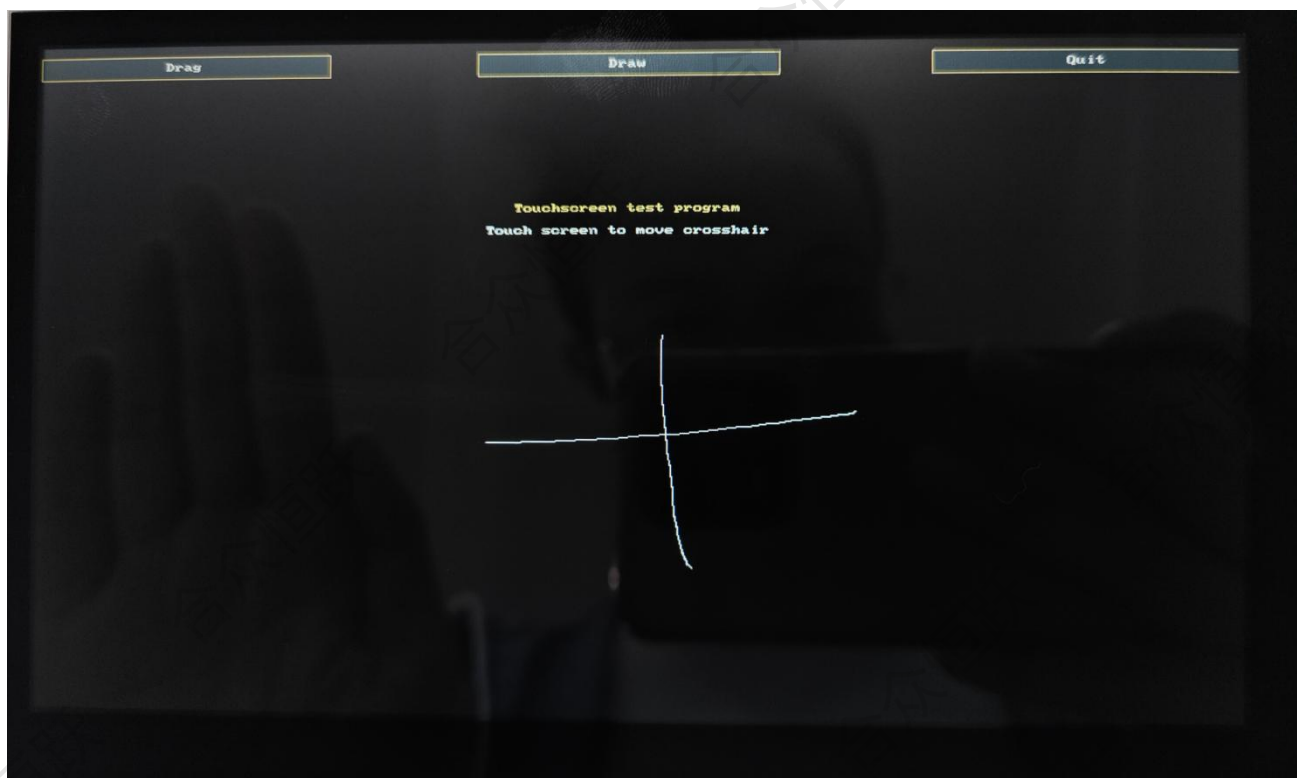


图 3.3-5 触摸屏校准

第 4 章 Qt 编译与部署

4.1 Qt 概述

Qt 是嵌入式应用中常用的图形界面开发框架，其跨平台性和强大的图形能力，使其成为在 Linux 系统上开发 UI 应用的主流方案之一。

4.2 Qt 交叉编译

1) 请参照第 2.1 节内容下载 qt5.14.2 的源码包，并将其放置于 Ubuntu 系统中的 /home/Qt_App/ 目录下。随后，执行以下命令对其进行解压。

```
hzh@ubuntu:~/Qt_App$ ls
qt-everywhere-src-5.14.2.tar.xz  tslib-1.16  tslib-1.16.tar.gz
hzh@ubuntu:~/Qt_App$ pwd
/home/hzh/Qt_App
hzh@ubuntu:~/Qt_App$ tar -xvf qt-everywhere-src-5.14.2.tar.xz
```

图 4.2-1 qt 解压

2) 进入 qt 源码目录，通过 vi 命令打开 qtbase/mkspecs/linux-aarch64-gnu-g++/qmake.conf 文件，写入如下内容对 qmake 进行配置。

```
MAKEFILE_GENERATOR      = UNIX
CONFIG                  += incremental
QMAKE_INCREMENTAL_STYLE = sublib
include(../common/linux.conf)
include(../common/gcc-base-unix.conf)
include(../common/g++-unix.conf)

QT_QPA_DEFAULT_PLATFORM = linuxfb

QMAKE_CFLAGS_RELEASE += -O2 -march=armv8-a -lts
QMAKE_CXXFLAGS_RELEASE += -O2 -march=armv8-a -lts
```



```
QMAKE_INCDIR += /home/hzhy/Qt_App/tslib-1.16/_install/include
QMAKE_LIBDIR += /home/hzhy/Qt_App/tslib-1.16/_install/lib
```

```
# modifications to g++.conf
```

```
QMAKE_CC          = aarch64-none-linux-gnu-gcc
QMAKE_CXX         = aarch64-none-linux-gnu-g++
QMAKE_LINK        = aarch64-none-linux-gnu-g++
QMAKE_LINK_SHLIB  = aarch64-none-linux-gnu-g++
```

```
# modifications to linux.conf
```

```
QMAKE_AR          = aarch64-none-linux-gnu-ar cqs
QMAKE_OBJCOPY     = aarch64-none-linux-gnu-objcopy
QMAKE_NM          = aarch64-none-linux-gnu-nm -P
QMAKE_STRIP       = aarch64-none-linux-gnu-strip
```

```
load(qt_config)
```

```
MAKEFILE_GENERATOR = UNIX
CONFIG             += incremental
QMAKE_INCREMENTAL_STYLE = sublib
```

```
include( ../common/linux.conf)
include( ../common/gcc-base-unix.conf)
include( ../common/g++-unix.conf)
```

```
QT_QPA_DEFAULT_PLATFORM = linuxfb
QMAKE_CFLAGS_RELEASE += -O2 -march=armv8-a -lts
QMAKE_CXXFLAGS_RELEASE += -O2 -march=armv8-a -lts
QMAKE_INCDIR += /home/hzhy/Qt_App/tslib-1.16/_install/include
QMAKE_LIBDIR += /home/hzhy/Qt_App/tslib-1.16/_install/lib
```

```
# modifications to g++.conf
```

```
QMAKE_CC          = aarch64-none-linux-gnu-gcc
QMAKE_CXX         = aarch64-none-linux-gnu-g++
QMAKE_LINK        = aarch64-none-linux-gnu-g++
QMAKE_LINK_SHLIB  = aarch64-none-linux-gnu-g++
```

```
# modifications to linux.conf
```

```
QMAKE_AR          = aarch64-none-linux-gnu-ar cqs
QMAKE_OBJCOPY     = aarch64-none-linux-gnu-objcopy
QMAKE_NM          = aarch64-none-linux-gnu-nm -P
QMAKE_STRIP       = aarch64-none-linux-gnu-strip
load(qt_config)
```

```
~qtbase/mkspecs/linux-aarch64-gnu-g++/qmake.conf" [已修改] 30 行 --83%--
```

图 4.2-2 qmake 配置

- 3) 创建安装目录 qt5.14.2，然后执行如下命令对 qt 进行配置。

```
./configure -prefix /home/hzhy/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2 -confirm-license -xplatform
linux-aarch64-gnu-g++ -opensource -verbose -skip qt3d -skip qtquickcontrols -skip qtwebchannel -
skip qtactiveqt -skip qtandroidextras -skip qtdeclarative -skip qtimageformats -skip qtmacextras -s
kip qtx11extras -skip qtxmlpatterns -skip qtconnectivity -skip qtdoc -skip qtgraphicaleffects -skip qt
location -skip qtmultimedia -skip qtsensors -skip qttools -skip qttranslations -skip qtwayland -skip
qtwebchannel -skip qtwebengine -skip qtwinextras -no-opengl -widgets -linuxfb -tslib -I/home/hzhy/
Qt_App/tslib-1.16/_install/include -L/home/hzhy/Qt_App/tslib-1.16/_install/lib -recheck-all
```

```
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2$ mkdir qt5.14.2
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2$ pwd
/home/hzhy/Qt_App/qt-everywhere-src-5.14.2
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2$ ./configure -prefix /home/hzhy/Qt_App/qt-everyw
here-src-5.14.2/qt5.14.2 -confirm-license -xplatform linux-aarch64-gnu-g++ -opensource -verbos
e -skip qt3d -skip qtquickcontrols -skip qtwebchannel -skip qtactiveqt -skip qtandroidextras -
skip qtdeclarative -skip qtimageformats -skip qtmacextras -skip qtx11extras -skip qtxmlpattern
s -skip qtconnectivity -skip qtdoc -skip qtgraphicaleffects -skip qtlocation -skip qtmultimedi
a -skip qtsensors -skip qttools -skip qttranslations -skip qtwayland -skip qtwebchannel -skip
qtwebengine -skip qtwinextras -no-opengl -widgets -linuxfb -tslib I/home/hzhy/Qt_App/tslib-1.1
6/_install/include -L/home/hzhy/Qt_App/tslib-1.16/_install/lib -recheck-all
```

图 4.2-3 Configure 配置

- 4) 看到下图信息说明配置成功。

```
SDL2 ..... no
Qt TextToSpeech:
  Flite ..... no
  Flite with ALSA ..... no
  Speech Dispatcher ..... no

Note: Also available for Linux: linux-clang linux-icc

WARNING: Cross compiling without sysroot. Disabling pkg-config

Qt is now configured for building. Just run 'make'.
Once everything is built, you must run 'make install'.
Qt will be installed into '/home/hzhy/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2'.

Prior to reconfiguration, make sure you remove any leftovers from
the previous build.
```

图 4.2-4 Configure 配置成功

- 5) 源码编译，make -j8（8 线程编译，根据自己电脑配置情况选择）。

```
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2$ make -j8
cd qtbase/ && ( test -e Makefile || /home/hzhy/Qt_App/qt-everywhere-src-5.14.2/qtbase/bin/qmak
e -o Makefile /home/hzhy/Qt_App/qt-everywhere-src-5.14.2/qtbase/qtbase.pro ) && make -f Makefi
```

图 4.2-5 源码编译

- 6) 将生成的文件打包安装。


```
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2$ make install
```

图 4.2-6 打包安装

7) 安装后的文件如图所示。

```
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2$ ls qt5.14.2/  
bin doc examples include lib mkspecs plugins
```

图 4.2-6 安装文件查看

至此 Qt 就交叉编译完成。

4.3 Qt 部署

1) 将上述编译安装后的文件放置到开发板的/opt 目录。

```
root@HZHY:/opt# ls /opt/qt5.14.2/  
bin doc examples include lib mkspecs plugins
```

图 4.3-1 文件传输

2) 通过 vi 命令打开/etc/profile，然后中增加如下配置。

```
export QTDIR=/opt/qt5.14.2/          #qt 交叉编译后文件  
export PATH=$QTDIR/bin:$PATH  
export LD_LIBRARY_PATH=$QTDIR/lib:$LD_LIBRARY_PATH  
export QT_PLUGIN_PATH=$QTDIR/plugins  
export QT_QPA_FONTDIR=/opt/qt5/fonts # 可选，设置字体路径  
  
# Qt 平台 & tslib 支持  
export QT_QPA_PLATFORM=linuxfb  
export QT_QPA_GENERIC_PLUGINS=tslib  
export QT_QPA_FB_DEVICE=/dev/fb0 # 可选，视需要
```

```
export QTDIR=/opt/qt5.14.2/  
export PATH=$QTDIR/bin:$PATH  
export LD_LIBRARY_PATH=$QTDIR/lib:$LD_LIBRARY_PATH  
export QT_PLUGIN_PATH=$QTDIR/plugins  
export QT_QPA_FONTDIR=/opt/qt5/fonts  
  
export QT_QPA_PLATFORM=linuxfb  
export QT_QPA_GENERIC_PLUGINS=tslib  
export QT_QPA_FB_DEVICE=/dev/fb0  
- /etc/profile [Modified] 47/47 100%
```

图 4.3-2 qt 环境配置

3) 执行如下命令使配置生效。

```
root@HZHY:/opt# source /etc/profile
```

图 4.3-3 配置生效

第 5 章 Qt 应用开发

5.1 概述

在完成 Qt 编译部署后，用户可使用 Qt Creator 导入或新建工程进行应用开发。本章将以 Qt 安装包中的示例项目为例，介绍在 Qt Creator 中加载、编译并部署应用到开发板的基本流程。

5.2 Qt Creator 使用

- 1) 在 ubuntu 打开 Qt Creator，其主界面如图所示，不同版本显示可能不同。



图 5.2-1 qtcreator 主界面

2) 点击工具栏的工具->外部->配置，弹出如下界面。

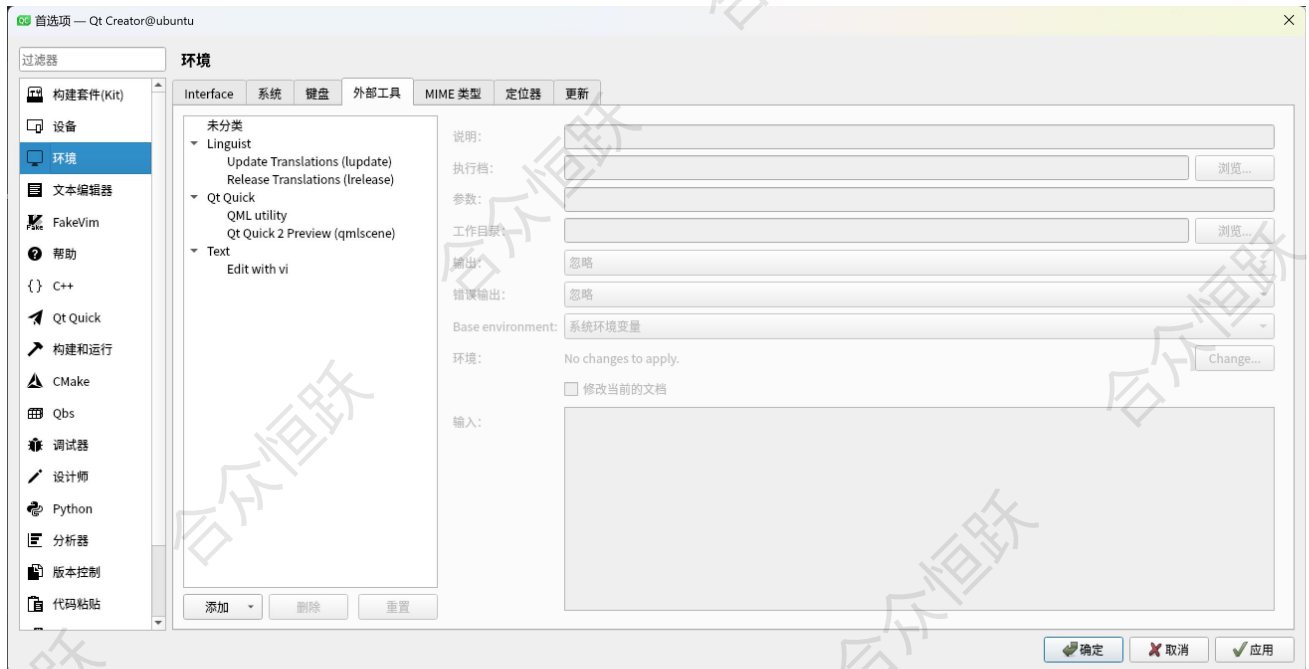


图 5.2-2 配置界面

3) 点击构建套件，开始对 QT 进行配置。



图 5.2-3 构建套件

4) 配置 GCC，点击编译器->添加->GCC->C。

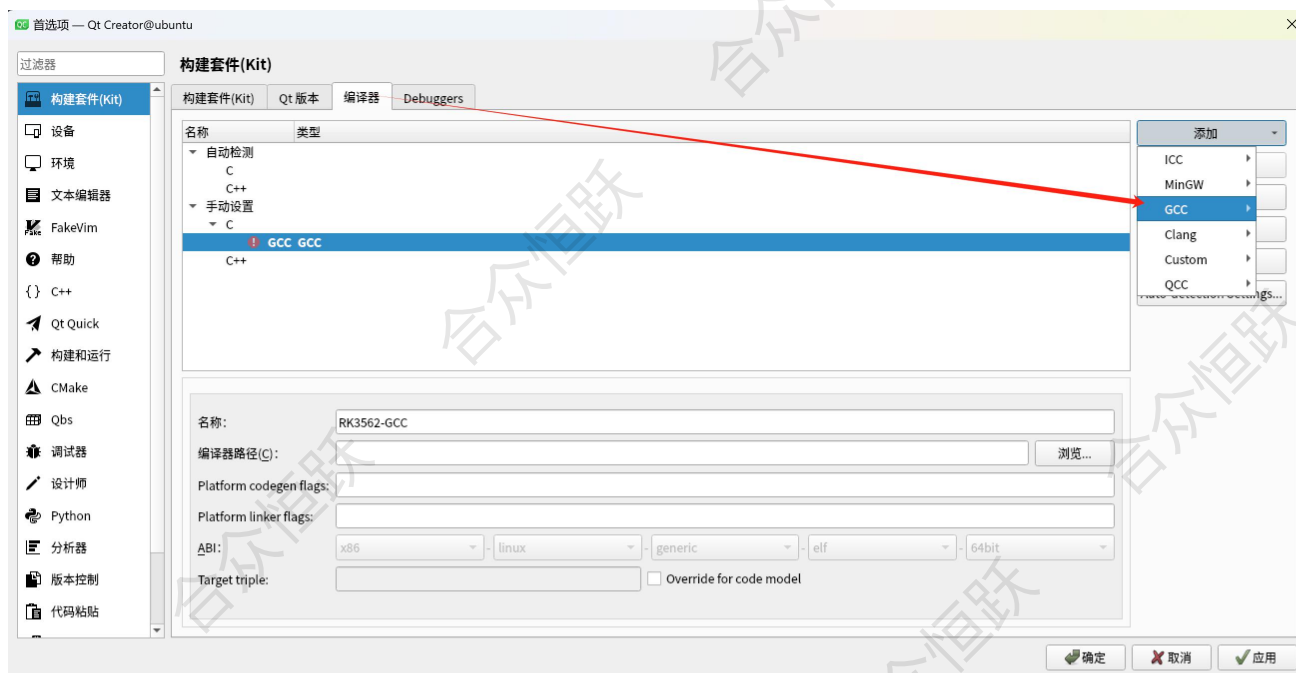


图 5.2-4 gcc 配置

5) 编写名称和编译器路径，路径为《Linux 开发环境搭建手册》中配置的路径，这里配置的路径如下。

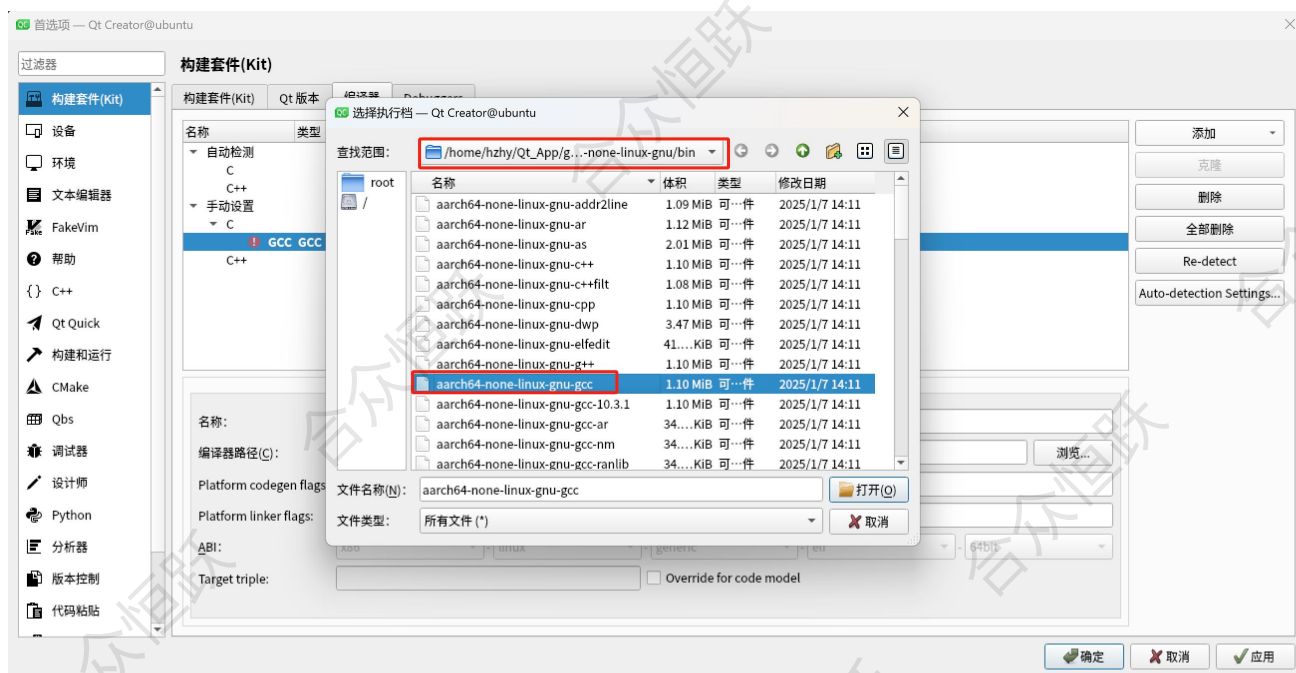


图 5.2-5 gcc 配置

6) gcc 配置完成后点击应用。

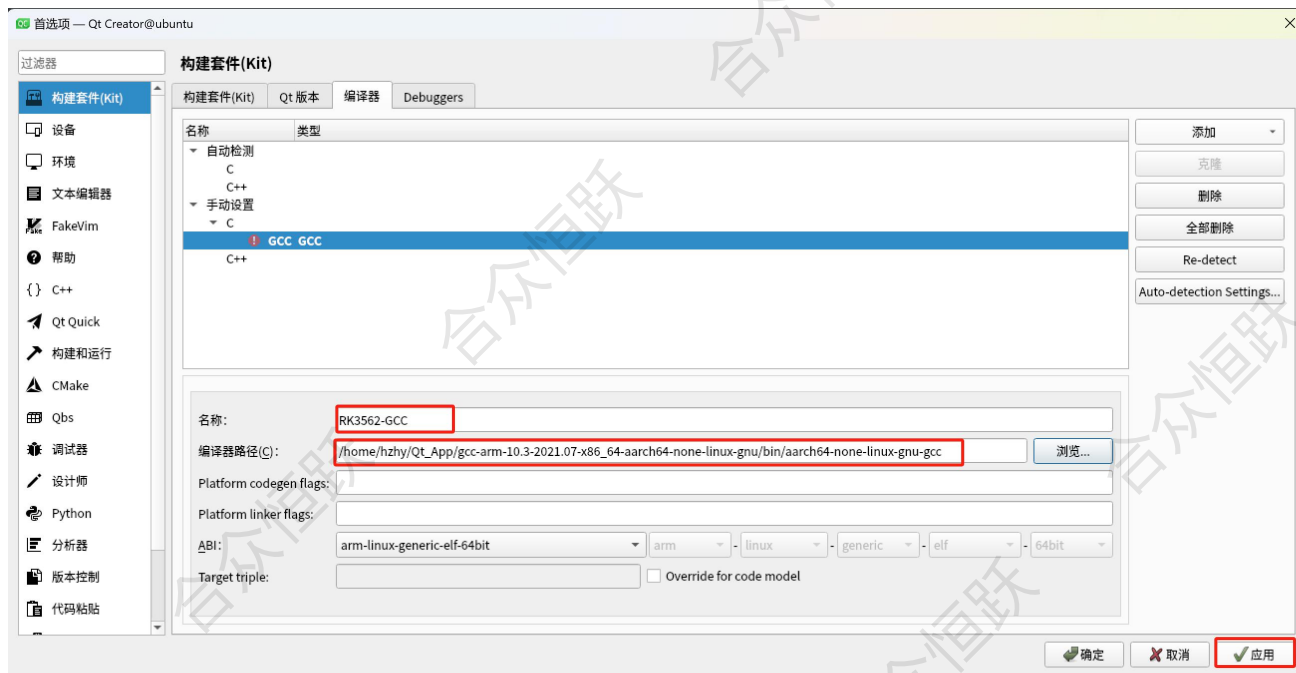


图 5.2-6 gcc 配置

7) 配置 g++, 点击编译器->添加->GCC->C++。

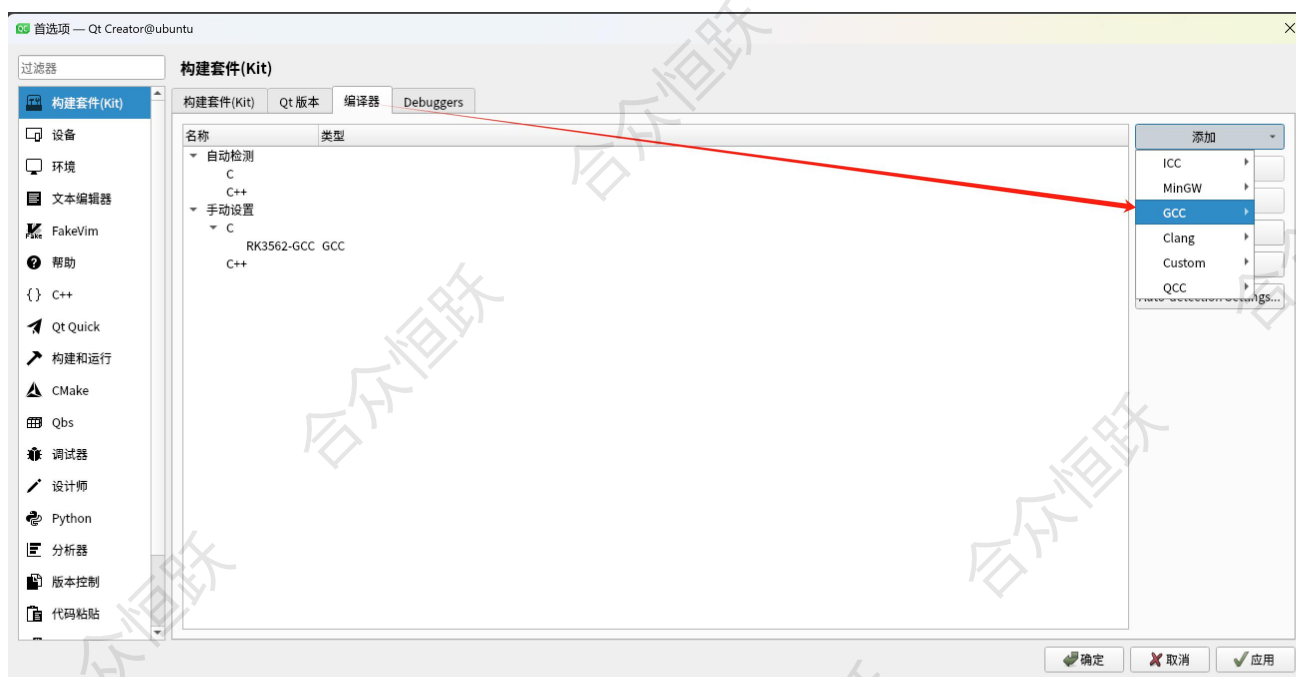


图 5.2-7 g++配置

8) 编写名称和编译器路径, 路径为《Linux 开发环境搭建手册》中配置的路径, 这里配置的路径如下。

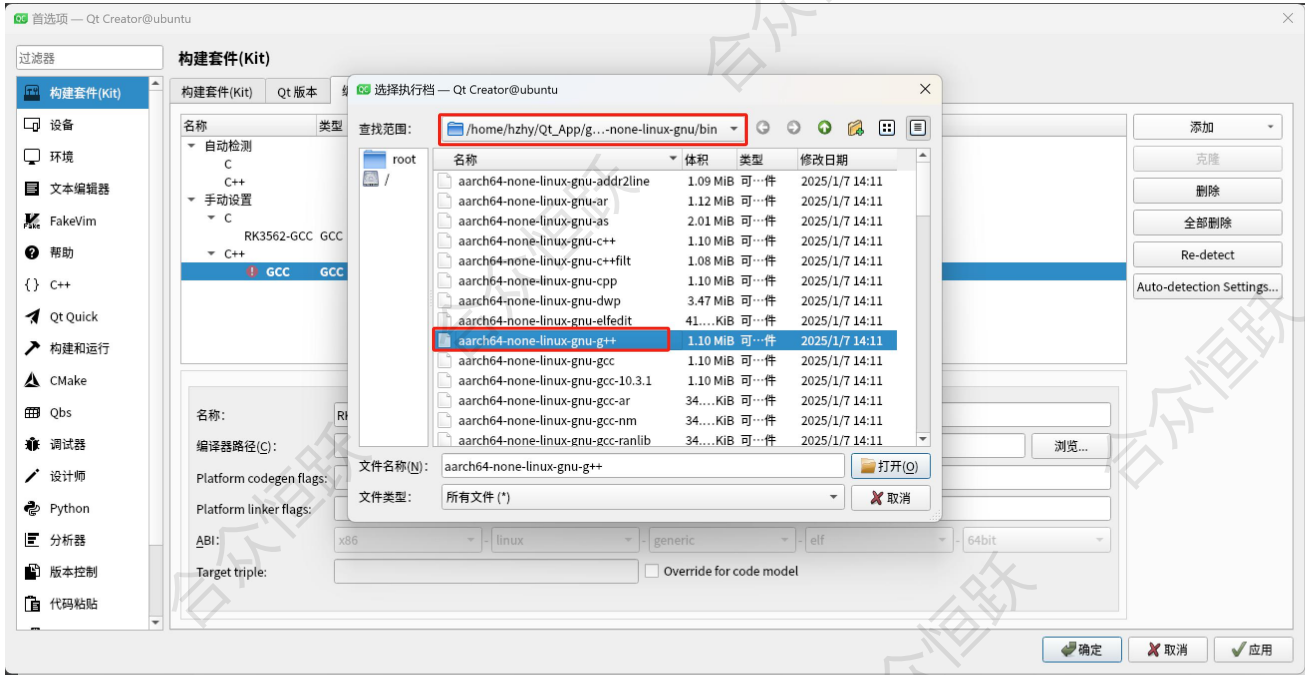


图 5.2-8 g++配置

9) g++配置完成后点击应用。

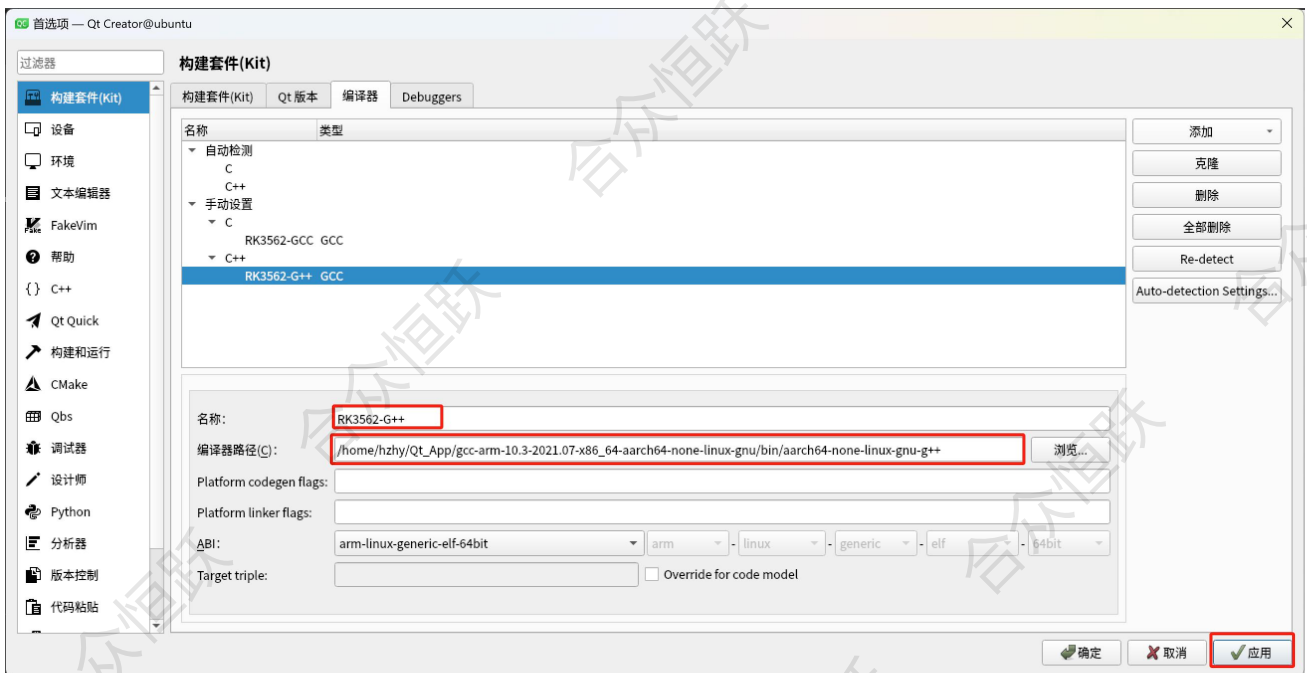


图 5.2-9 g++配置

10) 配置 qmake, qmake 为 qt 交叉编译时生成的, 在 ubuntu 中的路径如下。

```
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2/bin$ p
wd
/home/hzhy/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2/bin
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2/bin$ l
s
canbusutil      qdbuscpp2xml    qmake           repc            uic
fixqt4headers.pl qdbusxml2cpp    qvkgen          syncqt.pl
moc             qlalr           rcc             tracegen
```

图 5.2-10 qmake 路径

11) 点击 Qt 版本->添加。

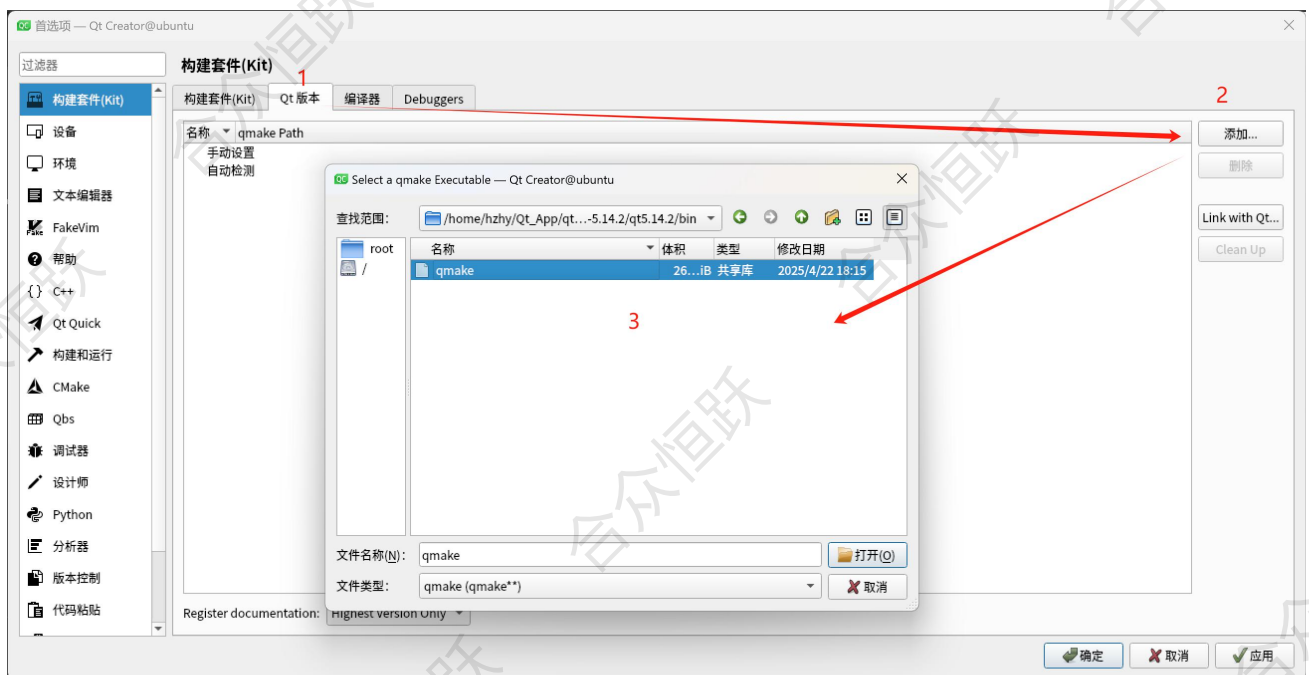


图 5.2-11 qmake 添加

12) 添加完成之后，点击应用。

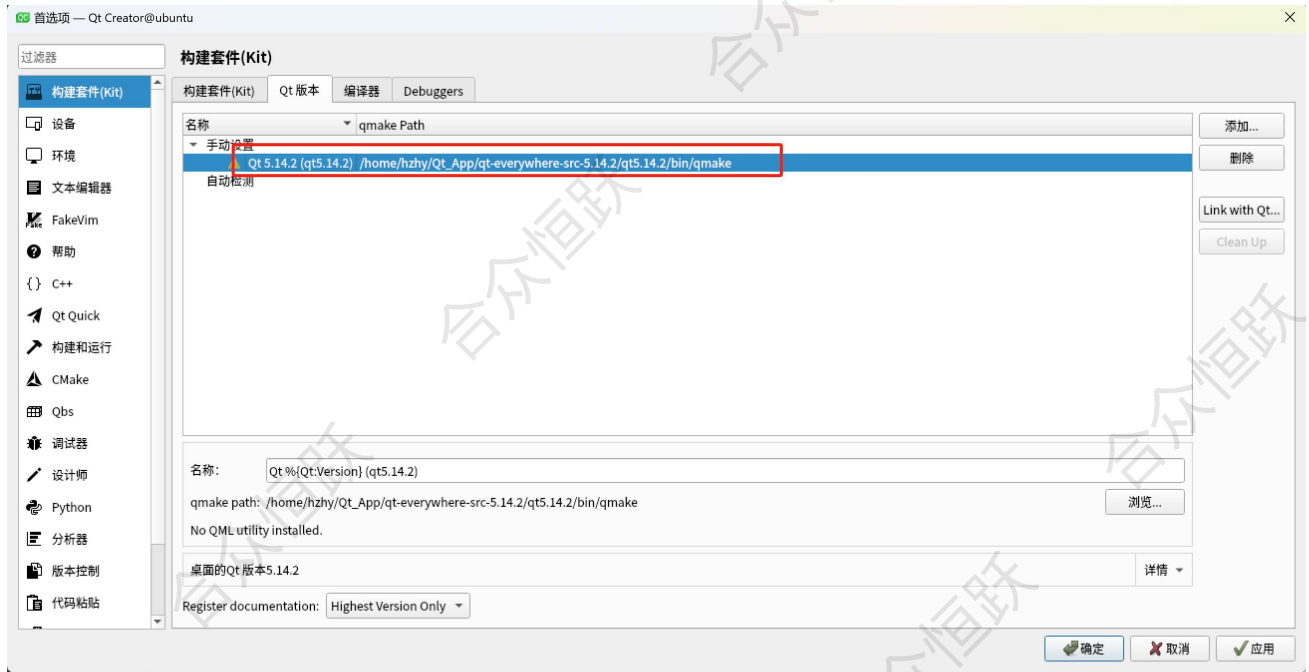


图 5.2-12 qmake 应用

13) 使用添加的编译器和 qmake 构建套件。

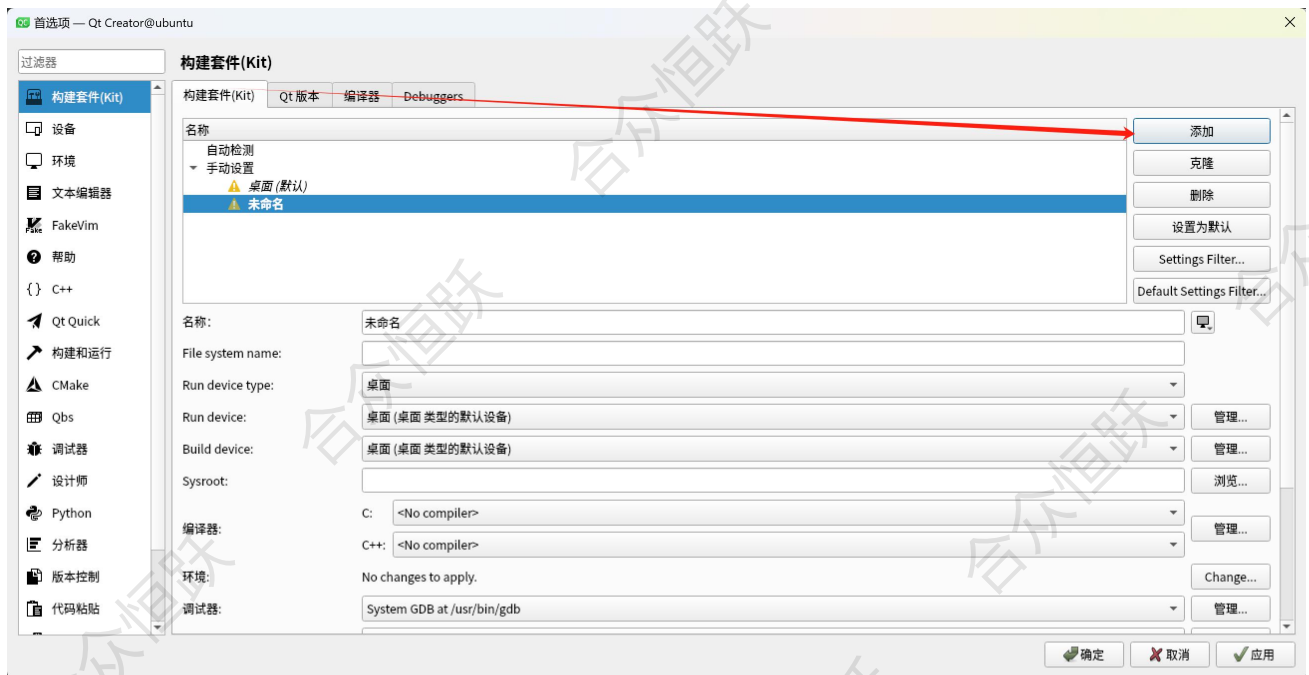


图 5.2-13 构建套件

14) 给构建套件取一个名称，然后依次选择编译器和 Qt 版本，然后点击确定。



图 5.1-14 构建套件

15) 点击打开项目，打开一个已经存在的项目。



图 5.1-15 打开项目

16) 这里我们使用 qt5.14.2 中的 gui 示例项目进行演示，里面还有很多其它示例，路径如下。

```
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2$ ls  
bin doc examples include lib mkspecs plugins  
hzhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2$ pwd  
/home/hzhy/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2
```

图 5.1-16 示例路径

17) 打开上述示例项目中的 gui/examples/gui/analogclock/analogclock.pro 工程。

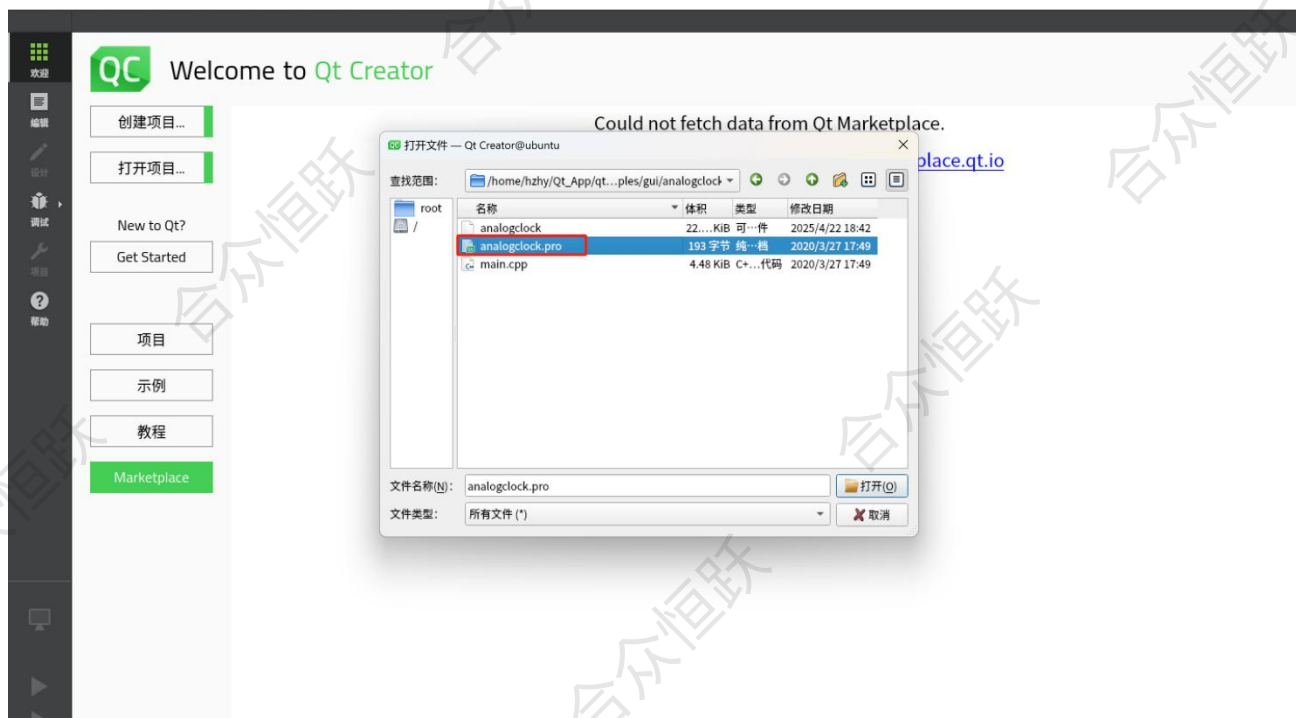


图 5.1-17 选中项目

18) 选中上面构建好的套件，然后点击配置项目。

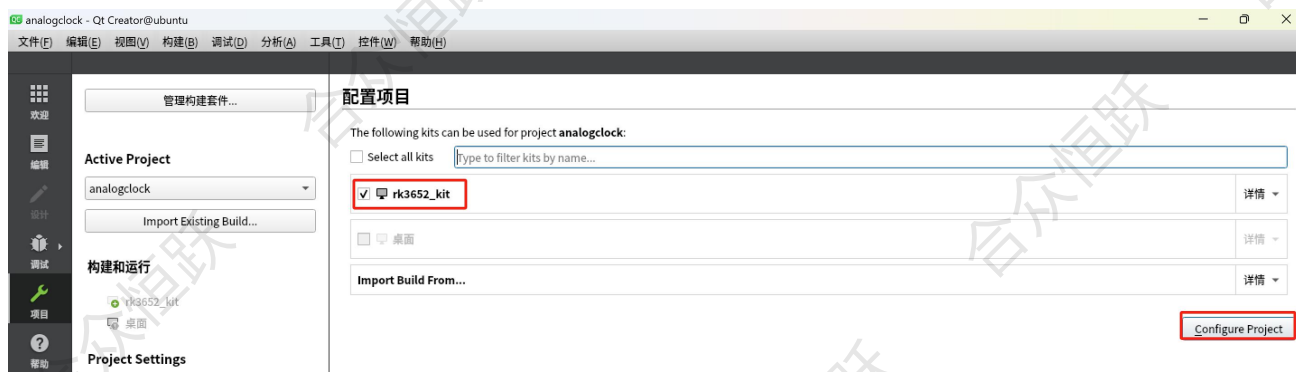


图 5.1-18 配置项目

19) 配置完成，在这里可以对项目进行编译、修改。

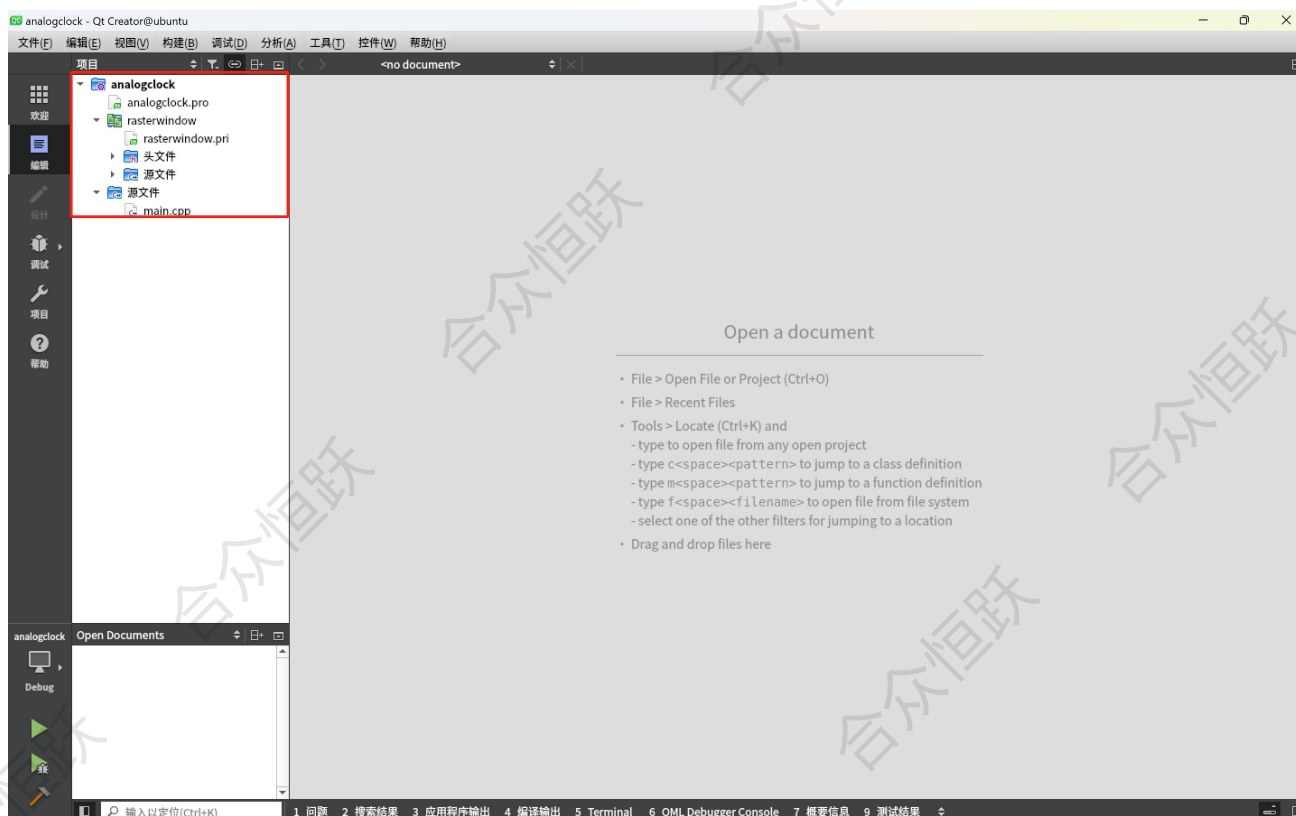


图 5.1-19 配置项目

20) 因为在编译 qt 时，该示例工程已经编译完成，所以这里删除掉已经生成的可执行文件。

```
es/gui/analogclock$ rm analogclock
hzy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2/example$ ls
analogclock.pro analogclock.pro.user main.cpp
```

图 5.1-20 删除文件

21) 选中项目，点击如下图标开始构建项目。

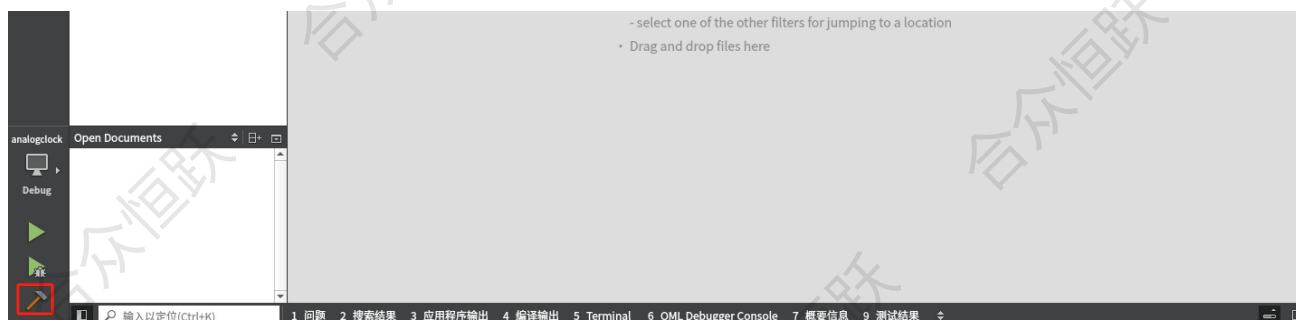


图 5.1-21 构建项目

22) 构建完成后，在 ubuntu 中再次查看生成的可执行文件。


```
es/gui/analogclock$ ls
analogclock          Makefile
analogclock.pro      moc_predefs.h
analogclock.pro.user moc_rasterwindow.cpp
main.cpp             moc_rasterwindow.o
main.o              rasterwindow.o
hzhhy@ubuntu:~/Qt_App/qt-everywhere-src-5.14.2/qt5.14.2/exampl
```

图 5.1-22 文件查看

到这程序就构建完成，并且已经生成了用于 arm 开发板的可执行程序，其它示例编译方法和该示例一致，有兴趣可自行研究。

5.3 Qt 示例演示

- 1) 将编译完成的可执行程序传输至开发板，并赋予其执行权限。

```
root@HZHY:~# ls
analogclock
root@HZHY:~# chmod a+x analogclock
root@HZHY:~# ls -l analogclock
-rwxr-xr-x 1 root root 23328 Apr 28 02:33 analogclock
```

图 5.3-1 文件传输

- 2) 在板子上运行 qt 应用程序。

```
root@HZHY:~# ./analogclock
```

图 5.3-2 程序执行

- 3) 应用程序启动后，屏幕上呈现以下界面：。



图 5.3-3 程序执行